



Redes Neuronales Bien Entrenadas

Sprint 10 · Una guía completa desde la neurona biológica hasta los optimizadores modernos. Aprenderás a construir, entrenar y depurar redes neuronales con sólidas bases matemáticas e intuición práctica.

SPRINT 10 · S10.1

Lo que aprenderás en este sprint

Recorreremos todos los fundamentos necesarios para entender y construir redes neuronales desde cero, con énfasis en la intuición matemática y la implementación práctica.

01

Neurona biológica → artificial

Origen e inspiración del modelo
computacional

02

Perceptrón y activaciones

El bloque básico de construcción

03

Forward pass y pérdida

Cómo fluye la información y cómo medimos
el error

04

Backpropagation y optimizadores

El motor del aprendizaje: gradientes y actualización de pesos

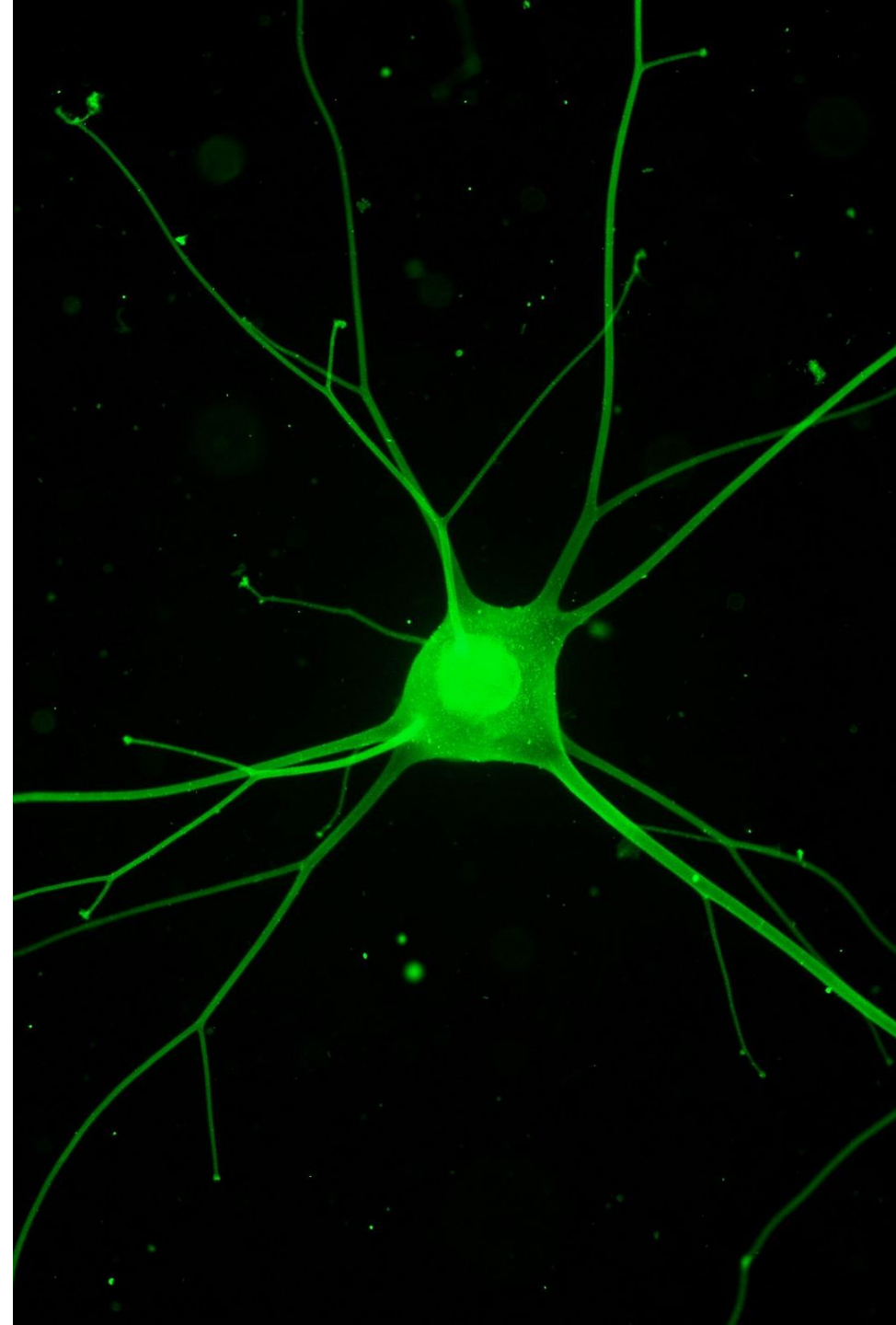
05

Implementación y limitaciones

Código real en NumPy y errores frecuentes

De la Neurona Biológica a la Artificial

Las redes neuronales artificiales se inspiran —con libertades creativas— en el funcionamiento del cerebro humano.



Neurona biológica vs. neurona artificial

Neurona biológica

- **Dendritas:** reciben señales de otras neuronas
- **Soma:** integra las señales entrantes
- **Axón:** transmite la señal de salida
- **Sinapsis:** conexión con la siguiente neurona; su fuerza varía con el aprendizaje
- La neurona «dispara» solo si el estímulo supera un umbral

Neurona artificial

- **Entradas $x_1...x_n$:** valores numéricos (píxeles, características, etc.)
- **Pesos $w_1...w_n$:** análogos a la fuerza sináptica
- **Sesgo b :** desplaza el umbral de activación
- **Combinación lineal:** $z = \sum w_i x_i + b$
- **Función de activación $f(z)$:** decide si y cómo «dispara»

❏ La analogía biológica es útil como motivación, pero la neurona artificial es en esencia un **regresor lineal con no-linealidad aplicada encima**.

El Perceptrón

El perceptrón, propuesto por Frank Rosenblatt en 1958, es el modelo más simple de neurona artificial. Es el ancestro directo de todas las redes profundas modernas.

Funcionamiento

Dado un vector de entrada $\mathbf{x}$, el perceptrón calcula una suma ponderada y aplica una función de activación escalonada:

$$\hat{y} = \text{sign} \left(\sum_i w_i x_i + b \right)$$

Si la salida es correcta, los pesos no cambian. Si no, se actualizan proporcionalmente al error. Este proceso se repite hasta convergencia.

Limitación histórica

Minsky y Papert (1969) demostraron que el perceptrón simple **no puede aprender la función XOR**, lo que desaceleró la investigación durante años. La solución: apilar capas.

Entradas

$x_1, x_2, \dots, x_n$ — valores reales

Pesos

$w_1, w_2, \dots, w_n$ — parámetros aprendidos

Salida

$\hat{y} \in \{-1, +1\}$ — clasificación binaria

Funciones de Activación

Las funciones de activación introducen **no-linealidad** en la red. Sin ellas, apilar capas sería equivalente a una única transformación lineal, incapaz de aproximar funciones complejas.



Sigmoid $\sigma(z)$

Salida en (0,1). Útil para probabilidades, pero sufre *vanishing gradient* en redes profundas.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



ReLU(z)

La más usada en la práctica. Simple, eficiente y no sufre vanishing gradient para valores positivos.

$$\text{ReLU}(z) = \max(0, z)$$



Tanh(z)

Salida en (-1,1), centrada en cero. Mejor que sigmoid para capas ocultas, pero sigue teniendo vanishing gradient.

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



Softmax

Convierte un vector de scores en probabilidades que suman 1. Estándar en la capa de salida para clasificación multiclase.

$$\text{softmax}(z_j) = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

Capas y Arquitectura

Organizar neuronas en capas define el poder expresivo de la red.



Anatomía de una red neuronal multicapa



Capa de entrada

Una neurona por cada característica del dato. No aplica transformación: solo distribuye los valores.



Capas ocultas

Donde ocurre el aprendizaje real. Cada capa extrae representaciones más abstractas que la anterior.



Capa de salida

Produce la predicción final. Su diseño depende de la tarea: 1 neurona (regresión), k neuronas + softmax (clasificación).

¿Cuántas capas y neuronas?

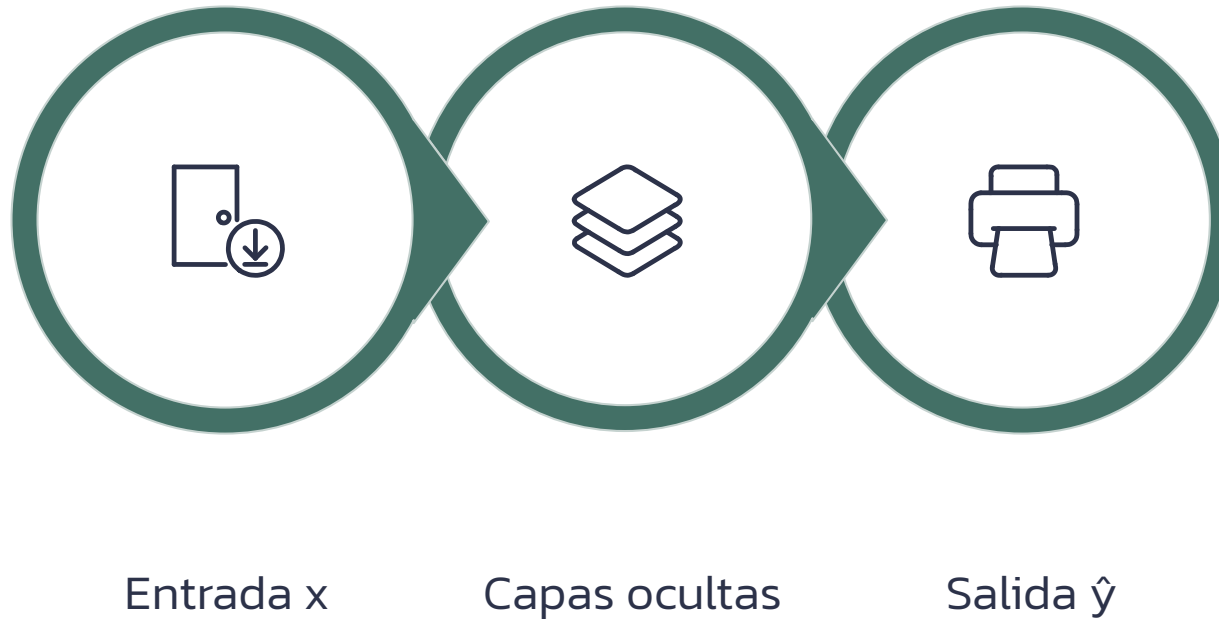
- Más capas → mayor capacidad de abstracción
- Más neuronas por capa → mayor capacidad por nivel
- Demasiada capacidad → riesgo de sobreajuste

Notación compacta

Una red **[784 → 128 → 64 → 10]** tiene tres capas con pesos, apropiada para clasificar dígitos MNIST.

El Forward Pass

El *forward pass* (pasada hacia delante) es el proceso mediante el cual un dato de entrada recorre la red capa a capa hasta producir una predicción. Es determinista: dados los pesos actuales, el resultado es siempre el mismo.



Para cada capa ℓ se calcula: $\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)} \cdot \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$ y luego $\mathbf{a}^{(\ell)} = \mathbf{f}(\mathbf{z}^{(\ell)})$. La salida de cada capa es la entrada de la siguiente. Este proceso es eficiente y completamente paralelizable en GPU.

Función de Pérdida

La función de pérdida (o *loss function*) cuantifica cuán lejos está la predicción $\hat{y}$ de la etiqueta real y . Es la brújula que guía el entrenamiento: el objetivo es **minimizarla**.

Regresión: MSE

Error cuadrático medio. Penaliza errores grandes de forma cuadrática, lo que hace que la red sea sensible a outliers.

$$\mathcal{L}_{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Clasificación: Cross-Entropy

Mide la divergencia entre la distribución predicha (softmax) y la distribución real (one-hot). Es la elección estándar en clasificación.

$$\mathcal{L}_{CE} = - \sum_c y_c \log(\hat{y}_c)$$

- ❏ La función de pérdida define implícitamente qué tipo de tarea estás resolviendo. Cambiarla cambia el comportamiento del entrenamiento de forma fundamental.

Backpropagation

El algoritmo que hace posible el aprendizaje profundo: calcular el gradiente de la pérdida respecto a cada peso de la red.



Intuición de Backpropagation

La *backpropagation* responde a la pregunta: «¿cuánto contribuyó cada peso al error final?» La respuesta es el gradiente $\partial L / \partial w$, que indica cuánto y en qué dirección debemos ajustar cada parámetro.

1 Forward pass

Calcula la predicción $\hat{y}$ y almacena las activaciones intermedias $a^{(l)}$ de cada capa. Estas se necesitarán en el paso siguiente.

2 Cálculo del error

Evalúa la función de pérdida $L(y, \hat{y})$ y calcula su derivada respecto a la salida: $\partial L / \partial \hat{y}$.

3 Propagación hacia atrás

Aplica la **regla de la cadena** capa a capa en orden inverso para obtener $\partial L / \partial W^{(l)}$ y $\partial L / \partial b^{(l)}$ en cada capa.

4 Actualización de pesos

Usa los gradientes para actualizar todos los parámetros de la red según la regla del optimizador elegido.

La Regla de la Cadena

La regla de la cadena del cálculo diferencial es el fundamento matemático de backpropagation. Permite descomponer la derivada de una función compuesta en un producto de derivadas simples.

Forma matemática

Si $L = L(a(z(w)))$, entonces:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Cada factor es una derivada «local» sencilla de calcular. Backprop los multiplica en orden inverso, de la salida hacia la entrada.

¿Por qué funciona en redes profundas?

- Las activaciones de cada capa son funciones diferenciables de la capa anterior
- El grafo computacional permite reutilizar gradientes ya calculados (*dynamic programming*)
- El coste total es proporcional al del forward pass: $O(\text{parámetros})$
- Frameworks como PyTorch implementan **autodiferenciación** automática

Gradiente Descendente y Variantes

Una vez calculados los gradientes, necesitamos una regla para actualizar los pesos. El gradiente descendente es el punto de partida, pero rara vez se usa en su forma pura.



En la práctica, el **mini-batch** de 32–256 muestras es el estándar: aprovecha la paralelización de GPU y produce gradientes suficientemente precisos para una convergencia estable.



Optimizadores

Los optimizadores definen cómo se actualiza cada peso a partir del gradiente. Son el motor del entrenamiento.

SGD: Stochastic Gradient Descent

Regla de actualización

La actualización de cada parámetro θ en cada paso t es simplemente:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} \mathcal{L}$$

Donde η es el **learning rate**, el hiperparámetro más importante del entrenamiento.

SGD con momento

Acumula un «impulso» de los gradientes anteriores para suavizar el recorrido y escapar de mínimos locales:

$$v_{t+1} = \mu v_t + \nabla \mathcal{L} \quad \theta_{t+1} = \theta_t - \eta v_{t+1}$$

Donde $\mu \approx 0.9$ es el coeficiente de momento.

Ventajas e inconvenientes de SGD

✓ Ventajas

- Simple y bien comprendido teóricamente
- Generaliza bien con el learning rate correcto
- Memoria mínima: solo almacena pesos y gradientes

⚠ Inconvenientes

- Sensible al learning rate: muy alto diverge, muy bajo es lento
- Requiere *learning rate scheduling* manual
- No adapta el paso a cada parámetro individualmente

Adam: Adaptive Moment Estimation

Adam (Kingma & Ba, 2015) combina las ideas de momento y de tasas de aprendizaje adaptativas por parámetro. Es el optimizador por defecto en la mayoría de proyectos de deep learning.

Algoritmo en 3 pasos

1. Primer momento (media exponencial de gradientes):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla \mathcal{L}$$

2. Segundo momento (media de gradientes al cuadrado):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla \mathcal{L})^2$$

3. Actualización con corrección de sesgo:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Hiperparámetros por defecto

$$\eta = 0.001$$

Learning rate inicial

$$\beta_1 = 0.9$$

Decaimiento del primer momento

$$\beta_2 = 0.999$$

Decaimiento del segundo momento

$$\epsilon = 1e-8$$

Estabilidad numérica

SGD vs. Adam: ¿Cuándo usar cada uno?

SGD

VELOCIDAD

Más lento.



GENERALIZACIÓN

A menudo superior en visión.



SENSIBILIDAD

Alta.



ADAPTIVIDAD

Tasa de aprendizaje uniforme.
uniforme.



USO RECOMENDADO

CNNs y tareas de visión.



ADAM

VELOCIDAD

Más rápido.



GENERALIZACIÓN

Aceptable, similar.



SENSIBILIDAD

Baja, más robusto.



ADAPTIVIDAD

Tasa de aprendizaje
adaptativa por parámetro.



USO RECOMENDADO

NLP, transformers,
prototipado rápido.



 Regla práctica: empieza con **Adam** para iterar rápido. Cambia a **SGD con momento y scheduler** si necesitas expresar el último punto de precisión en un modelo de visión.

Ejemplo en NumPy: Red Neuronal desde Cero

Implementemos una red $[2 \rightarrow 4 \rightarrow 1]$ para resolver XOR, el problema que derrotó al perceptrón simple. Toda la magia en menos de 30 líneas.

```
import numpy as np

# Activaciones
def sigmoid(z): return 1/(1+np.exp(-z))
def sigmoid_grad(a): return a*(1-a)

# Datos: XOR
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])

# Inicialización de pesos
np.random.seed(42)
W1 = np.random.randn(2,4)*0.1
b1 = np.zeros((1,4))
W2 = np.random.randn(4,1)*0.1
b2 = np.zeros((1,1))

lr = 0.5
for epoch in range(10000):
    # Forward
    z1 = X @ W1 + b1
    a1 = sigmoid(z1)
    z2 = a1 @ W2 + b2
    a2 = sigmoid(z2)
    # Pérdida MSE
    loss = np.mean((y - a2)**2)
    # Backward
    dL_da2 = -2*(y-a2)/len(y)
    dL_dz2 = dL_da2 * sigmoid_grad(a2)
    dL_dW2 = a1.T @ dL_dz2
    dL_da1 = dL_dz2 @ W2.T
    dL_dz1 = dL_da1 * sigmoid_grad(a1)
    dL_dW1 = X.T @ dL_dz1
    # Actualización
    W2 -= lr * dL_dW2
    W1 -= lr * dL_dW1
    b2 -= lr * dL_dz2.sum(0)
    b1 -= lr * dL_dz1.sum(0)

print(a2.round(2)) # [[0.02],[0.97],[0.97],[0.03]]
```

Puntos clave del código

Inicialización pequeña

Pesos $\times 0.1$ para evitar saturación de sigmoid desde el inicio.

Forward almacena $a1$

Las activaciones intermedias son necesarias en el backward: no las descartes.

Gradientes = álgebra matricial

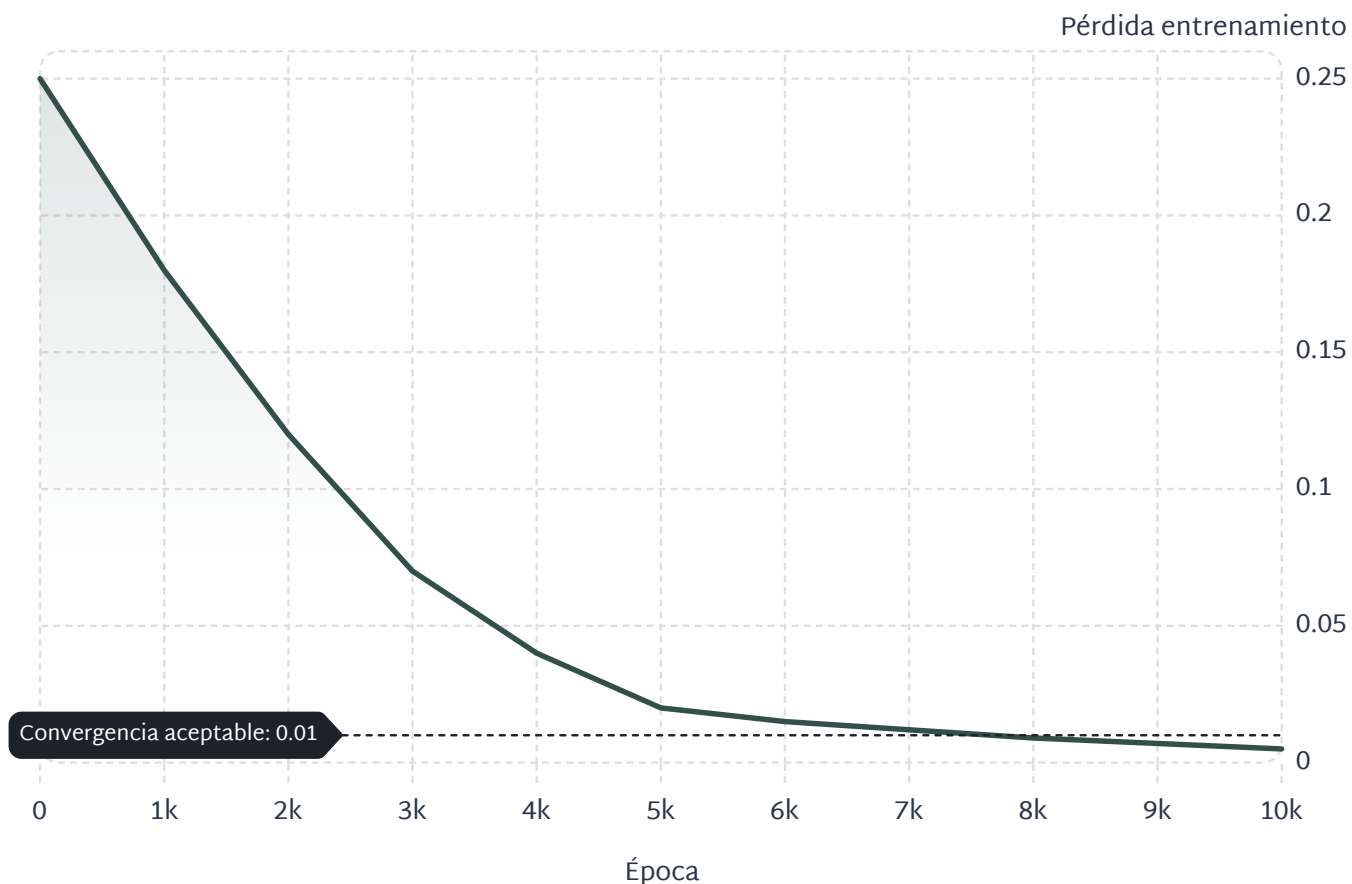
Toda la backprop son productos de matrices y multiplicaciones elemento a elemento.

Sin frameworks

Entender esto es el requisito previo para usar PyTorch o TensorFlow con criterio.

¿Qué ocurre durante el entrenamiento?

Visualizar cómo evoluciona la pérdida a lo largo de los epochs es esencial para diagnosticar problemas de entrenamiento.



Cómo leer la curva de pérdida

- **Descenso suave:** entrenamiento saludable
- **Plateau largo:** learning rate demasiado bajo o saturación
- **Oscilaciones:** learning rate demasiado alto
- **Divergencia:** exploding gradients o bug en el código
- **Pérdida estanca alta:** red con poca capacidad o datos mal normalizados

Limitaciones de las Redes Neuronales Simples

Conocer las limitaciones es tan importante como conocer las capacidades. Una red mal configurada puede ser peor que un modelo lineal bien ajustado.

Vanishing & Exploding Gradients

En redes muy profundas, los gradientes se multiplican capa a capa. Si son <1 , se «desvanecen» hacia la entrada; si son >1 , «explotan». Soluciones: ReLU, normalización por lotes, inicialización cuidadosa (Xavier, He).

Sobreajuste (Overfitting)

Una red con demasiados parámetros memoriza el conjunto de entrenamiento y no generaliza. Soluciones: regularización L1/L2, dropout, early stopping, aumento de datos.

Sensibilidad a la inicialización

Pesos inicializados a cero $\rightarrow$ todas las neuronas aprenden lo mismo (simetría). Inicialización aleatoria mal escalada $\rightarrow$ saturación o gradientes explosivos desde el inicio.

Coste computacional

Redes profundas requieren GPU y grandes conjuntos de datos para entrenar eficientemente. Las redes densas (fully-connected) escalan mal con datos de alta dimensionalidad (imágenes, audio).

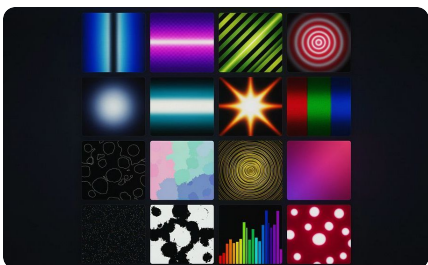
Problemas Frecuentes y Cómo Diagnosticarlos

El diagnóstico rápido es una habilidad esencial. Estos son los síntomas más comunes y sus remedios.

Síntoma	Causa probable	Solución recomendada
Pérdida no baja	Learning rate muy bajo o bug en backprop	Aumentar η ; hacer gradient check numérico
Pérdida oscila / diverge	Learning rate muy alto	Reducir η ; usar gradient clipping
Train loss baja, val loss sube	Sobreajuste	Dropout, L2, más datos, arquitectura más pequeña
Neuronas muertas (Dead ReLU)	Pesos negativos grandes con ReLU	Usar Leaky ReLU o He initialization
NaN en la pérdida	Log(0) o overflow numérico	Añadir ϵ ; clipear activaciones; reducir lr

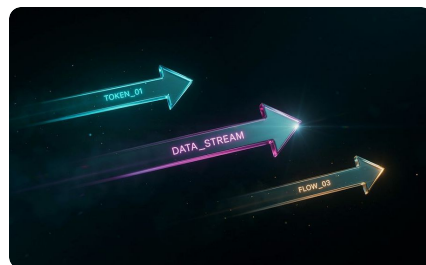
Más allá de las Redes Densas

Las redes fully-connected son el punto de partida, pero la arquitectura correcta depende del tipo de dato y de la tarea. Aquí empieza el zoo de arquitecturas.



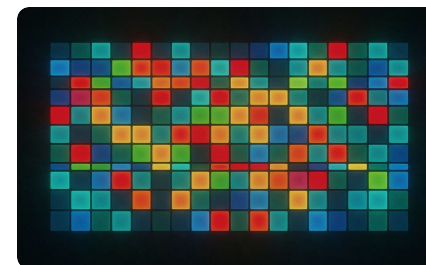
CNN

Redes Convolucionales. Explotan la estructura espacial de imágenes mediante filtros compartidos. Base de la visión por computador moderna.



RNN / LSTM

Redes Recurrentes. Procesan secuencias con memoria interna. Superadas por transformers en NLP, pero aún relevantes en series temporales.

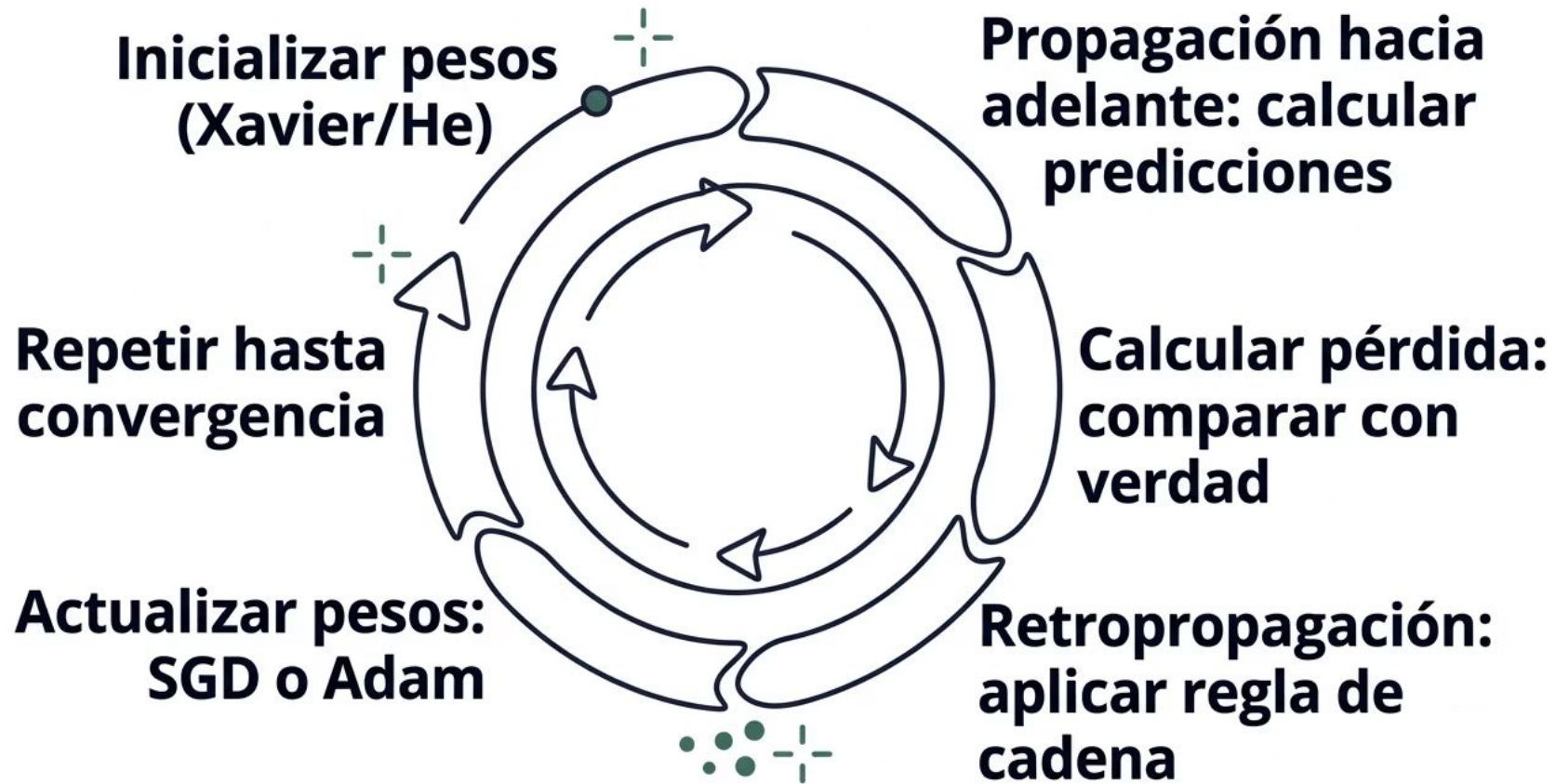


Transformers

Atención multi-cabeza. Arquitectura dominante en NLP (BERT, GPT) y visión (ViT). Escala excepcionalmente bien con datos y cómputo.

Resumen: El Ciclo de Entrenamiento

Todo el sprint en un único flujo. Comprender este ciclo en profundidad es el fundamento sobre el que se construye cualquier especialización en deep learning.



Próximos Pasos

Has adquirido los fundamentos sólidos. El camino sigue hacia arquitecturas especializadas, técnicas de regularización avanzadas y las mejores prácticas de entrenamiento a escala.



Practica

Reimplementa la red XOR en PyTorch usando `autograd`.
Compara con tu versión NumPy.



Profundiza

Deriva a mano la backprop de una capa con softmax + cross-entropy. Es el ejercicio más formativo del sprint.



Amplía

Añade regularización L2 y dropout a tu red NumPy. Observa cómo cambia la curva de pérdida en validación.



Lee

Deep Learning (Goodfellow et al.) caps. 6–8. Y el paper original de Adam (Kingma & Ba, 2015).



La mejor forma de entender backpropagation es **implementarla desde cero al menos una vez**. No hay atajo que sustituya esa experiencia.